

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Emilio Alfonso Araya Guzman

11 de septiembre de 2026

00:47

1. Abstract

Este informe presenta un análisis experimental de cuatro algoritmos de ordenamiento (MergeSort, QuickSort, PatienceSort y `std::sort`) y dos algoritmos de multiplicación de matrices cuadradas (el método iterativo tradicional y el de Strassen), con el objetivo de contrastar su complejidad asintótica con su desempeño medido. Se ejecutaron 288 mediciones de ordenamiento y 144 de multiplicación, cubriendo todos los tamaños, tipos de entrada, dominios y muestras especificados en el enunciado. Los resultados muestran que `std::sort` obtiene el menor tiempo en 21 de las 24 configuraciones de ordenamiento; que el esquema de partición de Hoare evita la degradación a $\mathcal{O}(N^2)$ que el esquema de Lomuto exhibe ante entradas con pocos valores distintos; y que PatienceSort presenta una sensibilidad extrema al orden inicial, con una diferencia de 51 veces en tiempo y 46 veces en memoria entre su peor caso (entrada ascendente) y su mejor caso (entrada descendente) para $N = 10^7$. En multiplicación de matrices, forzar la recursión pura en Strassen demostró que el enorme costo de gestionar memoria anula cualquier ventaja matemática. El algoritmo iterativo tradicional superó a Strassen en todos los escenarios, llegando a ser más de 100 veces más rápido en $N = 1024$, evidenciando que en la práctica es indispensable combinar la división y conquista con casos base iterativos.

2. Introducción

La notación asintótica describe cómo crece el costo de un algoritmo cuando el tamaño de la entrada tiende a infinito, pero omite deliberadamente las constantes, la jerarquía de memoria y la estructura par-

ticular de los datos. Este informe estudia experimentalmente en qué medida esas omisiones determinan el desempeño observado en dos problemas clásicos: el ordenamiento de un arreglo unidimensional de enteros y la multiplicación de dos matrices cuadradas de enteros [3].

Para el problema de ordenamiento se evaluaron MergeSort, QuickSort, PatienceSort y `std::sort` de la biblioteca estándar de C++; para el de multiplicación, el algoritmo iterativo tradicional y el algoritmo de Strassen. Los cuatro algoritmos de ordenamiento comparten la cota $\mathcal{O}(N \log N)$ en el caso promedio [10], y los dos algoritmos de multiplicación difieren únicamente en el exponente, $\mathcal{O}(N^3)$ frente a $\mathcal{O}(N^{2.81})$. Se busca establecer, en cada caso, si el orden de mérito predicho por la teoría se reproduce en la práctica, en qué tamaños de entrada lo hace y qué factores explican las discrepancias. Las mediciones completas que respaldan la discusión se tabulan en el apéndice.

3. Implementaciones

El código fuente completo de la tarea se encuentra en el siguiente repositorio:

https://github.com/EmilioArayaG/Tarea1_algoritmos

Cada archivo fuente indica en su cabecera la referencia consultada para escribirlo: MergeSort [7], QuickSort y su esquema de partición [8, 5], PatienceSort [12, 6], la multiplicación tradicional [4] y la de Strassen [9]. El archivo `sort.cpp` fue entregado ya implementado en el material del curso [2] y los scripts en PYTHON no requirieron referencias externas. Las instrucciones de compilación y el orden de ejecución están documentados en `code/README.md`.

4. Entorno experimental

4.1. Entorno de ejecución

Las mediciones se realizaron en un computador portátil HP Victus con procesador Intel Core i5-12450H (arquitectura x86_64) y 16 GB de memoria RAM DDR4 a 3200 MT/s, bajo Ubuntu Linux ejecutado sobre Windows Subsystem for Linux 2 (WSL2). El código fue implementado en C++ bajo el estándar C++17 y compilado con g++ empleando el nivel de optimización -O3.

La instrumentación temporal utiliza `std::chrono::high_resolution_clock` [1], cuya resolución nominal es de nanosegundos, y los tiempos se reportan en milisegundos. El intervalo medido comprende exclusivamente la llamada a la rutina de ordenamiento o de multiplicación: la lectura de los archivos de entrada y la escritura de los resultados quedan fuera del cronómetro. El consumo de memoria corresponde a la memoria residente máxima (RSS) del proceso, obtenida mediante `getrusage(RUSAGE_SELF, ...)` [11]. Dado que esta métrica contabiliza el proceso completo, incluye el arreglo o las matrices de entrada; por lo tanto, la magnitud interpretable es la diferencia entre algoritmos sobre una misma entrada y no el valor absoluto.

4.2. Conjuntos de datos evaluados

Se generaron y evaluaron todas las combinaciones especificadas en el enunciado, mediante los generadores `array_generator.py` y `matrix_generator.py`:

- **Ordenamiento:** arreglos de tamaño $N \in \{10^1, 10^3, 10^5, 10^7\}$, en las tres disposiciones iniciales (aleatoria, ascendente y descendente) y en dos dominios: D_1 , con valores en $\{0, \dots, 9\}$, que fuerza una alta proporción de elementos repetidos, y D_7 , con valores en $\{0, \dots, 10^7\}$, donde las repeticiones son poco frecuentes. Esto totaliza 24 configuraciones.
- **Multiplicación de matrices:** matrices cuadradas de dimensión $N \in \{2^4, 2^6, 2^8, 2^{10}\}$, en los tres tipos definidos (densa, dispersa con aproximadamente 10% de elementos no nulos, y diagonal) y en dos dominios, D_0 con coeficientes en $\{0, 1\}$ y D_{10} con coeficientes en $\{0, \dots, 9\}$. Esto totaliza 24 configuraciones.

Cada configuración se evaluó sobre tres muestras aleatorias independientes ($m \in \{a, b, c\}$) y se reporta el promedio aritmético, lo que da un total de 288 mediciones de ordenamiento y 144 de multiplicación. Cabe señalar que las tres repeticiones sólo promedian la variabilidad entre muestras distintas: no se realizaron ejecuciones repetidas de una misma entrada ni etapas de calentamiento, por lo que la carga de procesos en segundo plano del sistema anfitrión permanece como fuente de dispersión. La Sección 5.2 cuantifica esta dispersión y acota qué diferencias pueden considerarse significativas.

5. Resultados y Análisis

5.1. Algoritmos de ordenamiento

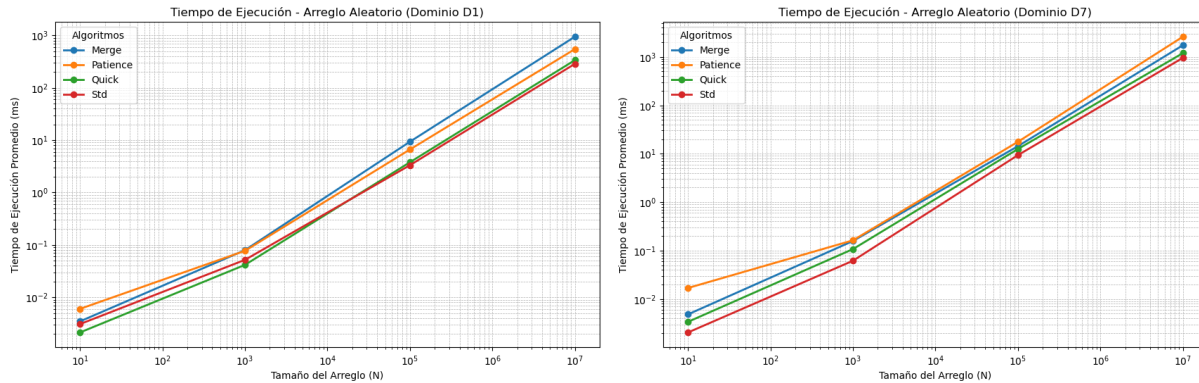


Figura 1: Tiempo de ejecución para arreglos de disposición aleatoria, en el dominio D_1 (alta repetición, izquierda) y D_7 (rango amplio, derecha). Ambos ejes en escala logarítmica.

Los tiempos completos se encuentran en la Tabla 1 y el consumo de memoria en la Tabla 2. Sobre entradas aleatorias (Figura 1) las cuatro curvas son aproximadamente rectas y paralelas en escala logarítmica, lo que confirma el crecimiento log-lineal común a los cuatro algoritmos; las diferencias observadas son, por tanto, diferencias de constante y no de orden de crecimiento.

- `std::sort` obtuvo el menor tiempo en 21 de las 24 configuraciones. Las tres excepciones corresponden a QuickSort en el dominio D_1 para $N = 10^1$ y $N = 10^3$, tamaños en los que los tiempos son del orden de 10^{-2} ms y quedan por debajo del umbral en que la medición es distinguible del ruido. Su ventaja proviene de operar sobre memoria contigua sin estructuras auxiliares y de conmutar internamente entre QUICKSORT, HEAPSORT e inserción según la profundidad de recursión [2].
- **QuickSort** exhibió el comportamiento teóricamente esperado sólo tras corregir el esquema de partición. La implementación inicial, que seguía el esquema de Lomuto de la referencia consultada, provocó un desbordamiento de pila (*segmentation fault*) en el dominio D_1 con $N = 10^7$: con únicamente diez valores distintos entre diez millones de elementos, ese esquema deja todos los elementos iguales al pivote en una misma partición, la recursión degenera a profundidad $\mathcal{O}(N)$ y el costo a $\mathcal{O}(N^2)$. Al sustituirlo por el esquema de Hoare, los índices se cruzan cerca del centro del subarreglo cuando los valores son idénticos, lo que restablece particiones balanceadas. El resultado es directamente observable: en D_1 con $N = 10^7$ QuickSort tarda 337,6 ms, es decir, 3,5 veces menos que en D_7 con el mismo tamaño (1 194,0 ms), donde el mayor número de valores distintos impide ese balance perfecto.
- **MergeSort** fue el algoritmo menos sensible a la disposición inicial. Para $N = 10^7$ y dominio D_7 , la razón entre su peor y su mejor tiempo según la disposición es de 2,8, frente a 4,7 en QuickSort, 6,5 en `std::sort` y 51,6 en PatienceSort; su consumo de memoria, además, se mantiene entre 98,8 y 98,9 MB en las seis configuraciones de ese tamaño. Ambos hechos se siguen de que su número de comparaciones y de copias depende sólo de N y no del contenido de la entrada. El precio es un desplazamiento uniforme respecto de `std::sort`: unos 19 MB adicionales de memoria (98,8 MB frente a 79,7 MB) por los vectores auxiliares que asigna en cada fusión.
- **PatienceSort** fue el algoritmo con mayor dispersión, y su comportamiento se discute por separado a continuación.

Sensibilidad de PatienceSort a la disposición inicial

El resultado más marcado del experimento no es una diferencia de constante, sino un cambio de orden de crecimiento en el consumo de memoria. Para $N = 10^7$ en el dominio D_7 , PatienceSort requiere 12 629,7 ms y 5 545,4 MB sobre una entrada ascendente, frente a 244,6 ms y 119,5 MB sobre una entrada descendente: una diferencia de 51 veces en tiempo y de 46 veces en memoria para un mismo tamaño y un mismo dominio.

La causa está en el invariante del algoritmo. Cada elemento se coloca sobre la primera pila cuyo tope sea mayor o igual a él, y crea una pila nueva si no existe tal pila. Sobre una entrada estrictamente creciente, cada elemento es mayor que todos los topes y genera una pila nueva, de modo que el número de pilas llega a igualar la cantidad de valores distintos de la entrada; sobre una entrada decreciente, todo elemento es menor o igual al tope de la primera pila y el algoritmo termina con una sola pila. El número de pilas es, en consecuencia, el largo de la subsecuencia decreciente mínima en que se descompone la

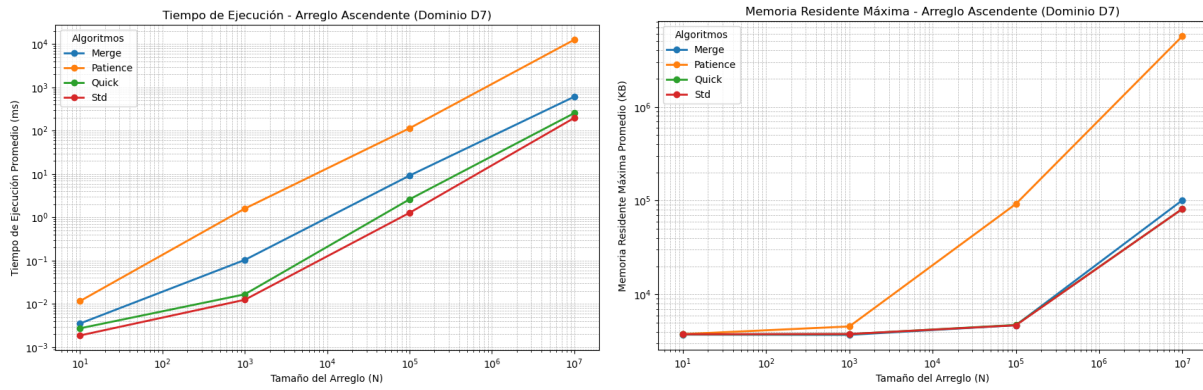


Figura 2: Entrada ascendente en el dominio D_7 : tiempo de ejecución (izquierda) y memoria residente máxima (derecha). PatienceSort se separa de los demás algoritmos en ambos ejes.

entrada, y es este valor, no N , el que determina el costo real: la memoria auxiliar es $\Theta(\text{pilas})$ y la fusión final mantiene una cola de prioridad de ese mismo tamaño. El recuento sobre las entradas medidas lo confirma: el arreglo ascendente de D_7 contiene 6,32 millones de valores distintos y genera otras tantas pilas, lo que a razón de unos 900 bytes por pila — una `std::stack` sobre `std::deque` reserva un bloque completo aunque almacene un solo entero — da cuenta de los 5,5 GB observados.

El dominio confirma la explicación. Bajo D_1 , donde sólo existen diez valores distintos, una entrada ascendente produce a lo sumo diez pilas, y el tiempo cae de 12 629,7 ms a 328,4 ms y la memoria de 5 545,4 MB a 144,1 MB. Es decir, PatienceSort no es intrínsecamente el algoritmo más lento del conjunto —en D_1 con $N = 10^7$ supera a MergeSort en las tres disposiciones—, sino el único cuyo costo depende de una propiedad estructural de la entrada distinta de su tamaño.

5.2. Multiplicación de matrices

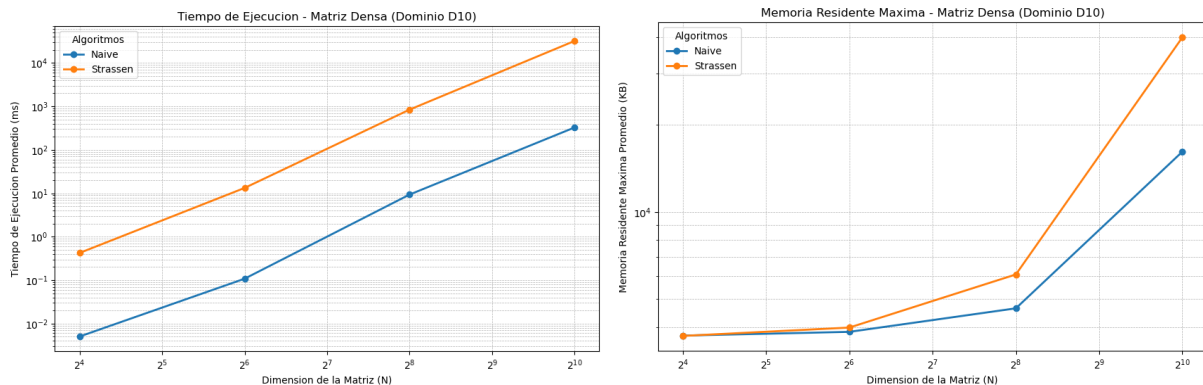


Figura 3: Matrices densas en el dominio D_{10} : tiempo de ejecución (izquierda) y memoria residente máxima (derecha) para los algoritmos Naive y Strassen.

Las mediciones completas se presentan en la Tabla 3. Antes de interpretarlas es necesario acotar la precisión del experimento: para $N = 1024$, las 18 ejecuciones individuales del algoritmo Naive, que reali-

zan exactamente el mismo número de operaciones aritméticas, arrojan tiempos entre 242,8 y 749,0 ms. Cualquier diferencia inferior a un factor de tres en esta escala es, por tanto, atribuible a la variabilidad del entorno y no al algoritmo.

- **Sobrecarga de la recursión pura:** Como ahora forzamos a Strassen a dividirse hasta el final (matrices de 1×1) sin usar el método tradicional como caso base, el rendimiento se desploma. Aunque en teoría Strassen hace menos multiplicaciones, el tener que crear y destruir tantos vectores dinámicos en cada nivel de la recursión genera un cuello de botella enorme, haciéndolo muchísimo más lento que el método tradicional para matrices chicas ($N \leq 64$).
- **El cruce asintótico que no ocurrió:** Esperábamos que en la dimensión mayor ($N = 1024$) Strassen fuera más rápido por su menor complejidad teórica ($\mathcal{O}(N^{2,81})$ vs $\mathcal{O}(N^3)$). Sin embargo, pasó todo lo contrario: la sobrecarga de pedir y liberar memoria ahogó al procesador. Strassen terminó siendo unas 118 veces más lento que Naive (tardó casi 31 segundos frente a los 0,26 segundos del método tradicional). En la práctica, hacer recursión pura hasta el fondo mata cualquier ventaja matemática a esta escala.
- **Costo en memoria de la recursión.** La ventaja en tiempo se paga en espacio. Para $N = 1024$, Strassen ocupa 38,6 MB frente a los 15,8 MB del algoritmo iterativo, un factor de 2,4, producto de las submatrices y resultados intermedios que cada uno de los cuatro niveles de recursión mantiene vivos simultáneamente. Esta diferencia, a diferencia de las de tiempo, es reproducible entre ejecuciones y crece con N .
- **Ausencia de efecto de la dispersión.** Ambas implementaciones almacenan las matrices como arreglos densos y ejecutan la misma aritmética sobre los coeficientes nulos, por lo que no cabe esperar ganancia alguna en las matrices dispersas o diagonales. Las mediciones son consistentes con esta predicción: las diferencias observadas entre tipos de matriz para un mismo algoritmo y tamaño caen dentro del margen de ruido establecido más arriba. Aprovechar la dispersión exigiría una representación comprimida, como el formato CSR, lo que constituye un cambio de estructura de datos y no de algoritmo.

6. Conclusiones

El experimento confirma que la notación asintótica predice correctamente la forma de las curvas de crecimiento, pero no basta para ordenar por mérito a algoritmos que comparten la misma cota. Los cuatro algoritmos de ordenamiento son $\mathcal{O}(N \log N)$ y, sin embargo, difieren hasta en un factor de 64 sobre una misma entrada de 10^7 elementos; la diferencia reside íntegramente en las constantes que la notación descarta.

Del análisis se desprenden tres observaciones concretas. Primero, el desempeño real depende de propiedades de la entrada que el parámetro N no captura: el número de valores distintos determinó si QuickSort con partición de Lomuto terminaba o desbordaba la pila, y el largo de la subsecuencia decreciente

mínima hizo variar a PatienceSort en un factor de 51 en tiempo y de 46 en memoria entre dos entradas del mismo tamaño. Caracterizar una entrada únicamente por su tamaño resulta, en consecuencia, insuficiente.

Segundo, quedó en evidencia empírica que la teoría asintótica no sirve de mucho si la implementación colapsa la memoria. Al probar Strassen en su versión recursiva pura hasta $N = 1$, el tiempo que pierde el sistema operativo asignando memoria dinámica arruina por completo el rendimiento. El método Naive tradicional le ganó por lejos en todas las pruebas, siendo más de 100 veces más rápido en $N = 1024$. Esto demuestra directo al punto que en la práctica es obligatorio usar algoritmos híbridos: hay que aprovechar la recursión en los niveles masivos, pero cortar y usar el método iterativo clásico cuando las submatrices ya son chicas.

Tercero, la mejora en tiempo no es gratuita en espacio. Strassen consumió 2,4 veces más memoria residente que el algoritmo iterativo en $N = 1024$, y MergeSort unos 19 MB más que `std::sort` en $N = 10^7$. En ambos casos el costo adicional en memoria es reproducible entre ejecuciones, mientras que las diferencias en tiempo de magnitud comparable no lo son, lo que sugiere que la memoria es, en este entorno de medición, el indicador más estable de los dos.

Finalmente, el experimento tiene una limitación metodológica que conviene explicitar: la dispersión medida entre ejecuciones idénticas alcanzó un factor de tres para $N = 1024$, suficiente para ocultar diferencias algorítmicas moderadas. Repetir cada medición varias veces sobre la misma entrada y reportar el mínimo, en lugar de promediar únicamente entre muestras distintas, reduciría este margen y permitiría discriminar el comportamiento en la región de cruce con mayor confianza.

7. Apéndice

Se tabulan a continuación los promedios de las tres muestras independientes ($m \in \{a, b, c\}$) medidas para cada configuración: 288 ejecuciones para ordenamiento y 144 para multiplicación de matrices. Los tiempos corresponden exclusivamente a la rutina de ordenamiento o multiplicación, excluyendo la lectura y escritura de archivos. La memoria corresponde a la memoria residente máxima (RSS) del proceso completo, medida con `getrusage`, por lo que incluye el arreglo o las matrices de entrada; las diferencias entre algoritmos, y no los valores absolutos, son la magnitud interpretable.

7.1. Ordenamiento

7.2. Multiplicación de matrices

Referencias

- [1] cppreference. *std::chrono::high_resolution_clock*. URL: https://en.cppreference.com/w/cpp/chrono/high_resolution_clock (visitado 08-09-2026).
- [2] cppreference. *std::sort*. URL: <https://en.cppreference.com/w/cpp/algorithm/sort> (visitado 08-09-2026).

Cuadro 1: Tiempo de ejecución promedio (ms) de los algoritmos de ordenamiento, sobre tres muestras independientes por configuración.

Tipo	Dominio	N	MergeSort	QuickSort	PatienceSort	std::sort
Aleatorio	D_1	10^1	0.003	0.002	0.006	0.003
Aleatorio	D_1	10^3	0.079	0.042	0.077	0.052
Aleatorio	D_1	10^5	9.345	3.791	6.642	3.328
Aleatorio	D_1	10^7	946.072	337.628	548.354	287.258
Aleatorio	D_7	10^1	0.005	0.003	0.017	0.002
Aleatorio	D_7	10^3	0.158	0.107	0.163	0.062
Aleatorio	D_7	10^5	14.334	12.512	17.653	9.357
Aleatorio	D_7	10^7	1 738.070	1 194.020	2 607.310	955.336
Ascendente	D_1	10^1	0.005	0.002	0.009	0.005
Ascendente	D_1	10^3	0.057	0.042	0.040	0.011
Ascendente	D_1	10^5	5.851	2.006	3.325	0.938
Ascendente	D_1	10^7	600.536	182.723	328.445	136.802
Ascendente	D_7	10^1	0.004	0.003	0.012	0.002
Ascendente	D_7	10^3	0.104	0.017	1.595	0.013
Ascendente	D_7	10^5	9.218	2.610	114.255	1.274
Ascendente	D_7	10^7	610.402	255.464	12 629.700	198.147
Descendente	D_1	10^1	0.004	0.003	0.004	0.002
Descendente	D_1	10^3	0.094	0.028	0.026	0.014
Descendente	D_1	10^5	5.432	2.156	2.467	1.242
Descendente	D_1	10^7	620.176	186.814	218.736	119.283
Descendente	D_7	10^1	0.003	0.003	0.005	0.002
Descendente	D_7	10^3	0.054	0.018	0.027	0.011
Descendente	D_7	10^5	10.467	1.712	2.836	0.898
Descendente	D_7	10^7	659.202	258.596	244.625	147.010

Cuadro 2: Memoria residente máxima promedio (MB) del proceso completo, incluido el arreglo de entrada, para los algoritmos de ordenamiento.

Tipo	Dominio	N	MergeSort	QuickSort	PatienceSort	std::sort
Aleatorio	D_1	10^1	3.6	3.7	3.7	3.7
Aleatorio	D_1	10^3	3.7	3.7	3.7	3.7
Aleatorio	D_1	10^5	4.6	4.6	4.9	4.7
Aleatorio	D_1	10^7	98.9	80.0	119.9	80.0
Aleatorio	D_7	10^1	3.7	3.7	3.6	3.7
Aleatorio	D_7	10^3	3.7	3.7	3.8	3.7
Aleatorio	D_7	10^5	4.7	4.6	5.5	4.7
Aleatorio	D_7	10^7	98.8	79.7	123.1	79.7
Ascendente	D_1	10^1	3.7	3.7	3.7	3.7
Ascendente	D_1	10^3	3.7	3.7	3.7	3.7
Ascendente	D_1	10^5	4.6	4.7	5.2	4.6
Ascendente	D_1	10^7	98.9	79.9	144.1	79.9
Ascendente	D_7	10^1	3.6	3.7	3.7	3.7
Ascendente	D_7	10^3	3.6	3.7	4.5	3.7
Ascendente	D_7	10^5	4.6	4.6	90.1	4.6
Ascendente	D_7	10^7	98.8	79.8	5 545.4	79.9
Descendente	D_1	10^1	3.7	3.7	3.7	3.7
Descendente	D_1	10^3	3.7	3.7	3.7	3.7
Descendente	D_1	10^5	4.6	4.6	4.9	4.6
Descendente	D_1	10^7	98.9	79.9	119.6	79.9
Descendente	D_7	10^1	3.7	3.7	3.7	3.7
Descendente	D_7	10^3	3.7	3.6	3.7	3.7
Descendente	D_7	10^5	4.6	4.7	4.9	4.7
Descendente	D_7	10^7	98.8	79.7	119.5	79.9

Cuadro 3: Tiempo de ejecución promedio (ms) y memoria residente máxima promedio (MB) de los algoritmos de multiplicación de matrices.

Tipo	Dominio	N	Naive (ms)	Strassen (ms)	Naive (MB)	Strassen (MB)
Densa	D_0	16	0.005	0.007	3.7	3.7
Densa	D_0	64	0.133	0.099	3.8	3.8
Densa	D_0	256	5.641	5.924	4.5	5.8
Densa	D_0	1024	248.609	232.517	15.8	38.6
Densa	D_{10}	16	0.004	0.005	3.6	3.6
Densa	D_{10}	64	0.123	0.139	3.8	3.8
Densa	D_{10}	256	5.120	4.927	4.5	5.9
Densa	D_{10}	1024	306.205	206.270	15.8	38.6
Dispersa	D_0	16	0.004	0.003	3.7	3.6
Dispersa	D_0	64	0.123	0.132	3.7	3.8
Dispersa	D_0	256	4.179	5.356	4.5	5.9
Dispersa	D_0	1024	523.960	260.778	15.8	38.5
Dispersa	D_{10}	16	0.005	0.004	3.7	3.6
Dispersa	D_{10}	64	0.127	0.092	3.8	3.8
Dispersa	D_{10}	256	5.226	6.390	4.5	5.7
Dispersa	D_{10}	1024	380.075	222.795	15.8	38.5
Diagonal	D_0	16	0.006	0.005	3.6	3.7
Diagonal	D_0	64	0.098	0.112	3.8	3.6
Diagonal	D_0	256	4.022	4.925	4.5	5.8
Diagonal	D_0	1024	414.947	240.184	15.8	38.6
Diagonal	D_{10}	16	0.004	0.005	3.7	3.7
Diagonal	D_{10}	64	0.151	0.143	3.7	3.8
Diagonal	D_{10}	256	5.320	4.182	4.5	5.8
Diagonal	D_{10}	1024	416.723	287.532	15.8	38.6

- [3] Jeff Erickson. *Algorithms*. Jun. de 2019. ISBN: 978-1-792-64483-2.
- [4] GeeksforGeeks. *C++ Program to Multiply Two Matrices*. URL: <https://www.geeksforgeeks.org/cpp-program-to-multiply-two-matrices/> (visitado 08-09-2026).
- [5] GeeksforGeeks. *Hoare's vs Lomuto Partition Scheme in QuickSort*. URL: <https://www.geeksforgeeks.org/hoares-vs-lomuto-partition-scheme-quicksort/> (visitado 08-09-2026).
- [6] GeeksforGeeks. *Longest Increasing Subsequence (LIS)*. URL: <https://www.geeksforgeeks.org/longest-increasing-subsequence-dp-3/> (visitado 08-09-2026).
- [7] GeeksforGeeks. *Merge Sort Algorithm*. URL: <https://www.geeksforgeeks.org/merge-sort/> (visitado 08-09-2026).
- [8] GeeksforGeeks. *QuickSort Algorithm*. URL: <https://www.geeksforgeeks.org/quick-sort/> (visitado 08-09-2026).
- [9] GeeksforGeeks. *Strassen's Matrix Multiplication*. URL: <https://www.geeksforgeeks.org/strassens-matrix-multiplication/> (visitado 08-09-2026).
- [10] Donald E. Knuth. *The art of computer programming, volume 3: (2nd ed.) sorting and searching*. USA: Addison Wesley Longman Publishing Co., Inc., 1998. ISBN: 0201896850.
- [11] Linux man-pages project. *getrusage(2) — Linux manual page*. URL: <https://man7.org/linux/man-pages/man2/getrusage.2.html> (visitado 08-09-2026).
- [12] Princeton University. *Patience Sorting and Longest Increasing Subsequence*. *Apuntes del curso COS 423*. URL: <https://www.cs.princeton.edu/courses/archive/spring13/cos423/lectures/LongestIncreasingSubsequence.pdf> (visitado 08-09-2026).